

Spring Boot — Lecture 3

Maven — Project Management Tool (Deep Dive)

1. JAR Files — Java Archives

Normal flow: **.java** → javac → **.class** (bytecode) → JVM runs it. A real project has hundreds of .java files. Sharing them one-by-one is impractical — that's why JAR exists.

Term	Full Form	What it is
JAR	Java Archive	A zip-like package of compiled .class files + resources. The standard Java sharing unit.
Library JAR	Non-executable JAR	Contains reusable classes/methods. No main(). Cannot run standalone.
App JAR	Executable JAR	Has main() + all dependency JARs embedded. Runs independently.

JAR contains: compiled .class files, resource files (.properties, images), package/folder structure, and a metadata file listing further dependencies.

Library JAR does NOT embed its own dependencies — it lists them in a metadata file. App JAR (e.g. Spring Boot) embeds everything — run with: `java -jar myapp.jar`

2. Library vs Application

Type	Has main()?	Runnable?	Embeds dependency JARs?
Library	No	No — used inside an app	No — metadata file lists them
Application	Yes	Yes — standalone	Yes — all embedded

3. Problems Without Maven

Problem	Detail
Manual downloads	Search Google for every JAR, download one-by-one. 100s of JARs = nightmare.
Version mismatches	Spring 7 works with Spring Boot 4, not 3. Wrong combo → runtime errors.
Transitive dependencies	Spring Boot depends on Tomcat → Tomcat depends on X → X depends on Y. You'd need to know all of them.
No sharing standard	Sharing your library requires a manual text file listing which JARs to download — very error-prone.
No folder standard	Each developer uses a different structure — team chaos.

4. Maven — What It Does For You

Maven Feature	Details
Folder structure	Enforces a standard directory layout — same for every developer, every IDE.
Compile code	Compiles all .java files to .class with one command.
Create JAR / WAR	Packages compiled code into a distributable JAR automatically.
Download dependencies	Name the library in pom.xml — Maven fetches it + all its transitive deps automatically.

5. Standard Maven Folder Structure

src/ main/ java/ ← All .java source files resources/ ← Non-Java: .properties, images, templates test/ java/ ← Unit test files (mirror main/ package structure) pom.xml ← Maven config (most important file) target/ ← Auto-generated: .class files + built JARs (never

```
commit to Git)
```

Package naming inside src/main/java follows groupId.artifactId structure. e.g. Group: com.example, Artifact: demo
→ folder path: com/example/demo/

target/ is auto-generated. Delete it with 'mvn clean'. Maven regenerates on next build.

6. pom.xml — Project Object Model

The most important Maven file. Stores complete project information.

XML Tag	Purpose
	POM format version — always 4.0.0
	Org unique ID — use reverse domain (e.g. in.coderarmyarmy or org.example)
	Your project/module name (e.g. demo, order-service)
	Project version (e.g. 1.0-SNAPSHOT while in development, 1.0.0 when released)
	jar (default, modern Spring Boot) or war (old-style web apps)
	Key-value config values reused across pom.xml (e.g. 21)
	List of all external libraries your project needs
	Inherit configuration from a parent POM — Spring Boot uses this heavily
	Maven plugins to extend lifecycle phases (code quality, reports, etc.)

GAV = Group + Artifact + Version — uniquely identifies any JAR worldwide. JAR filename = artifactId-version.jar (e.g. mysql-connector-j-9.5.0.jar)

Sample dependency entry:

```
com.mysql mysql-connector-j 9.5.0
```

7. Adding Dependencies — Step by Step

- Step 1: Go to mvnrepository.com — search for the library (e.g. 'mysql connector').
- Step 2: Pick a stable version — avoid the very latest; prefer one with many users listed.
- Step 3: Copy the Maven XML snippet — paste inside in pom.xml.
- Step 4: Click the Maven sync/reload button in IntelliJ (or run: mvn install).
- Step 5: Maven downloads the JAR + all transitive dependencies automatically.

Hibernate alone pulls in 15+ transitive JARs — Maven handles all of them. You never need to know about transitive deps yourself.

Dependency scope (optional):

Scope	When needed	Example
compile	Default — always available	Spring MVC, Hibernate
test	Only during test phase	JUnit, Mockito
provided	Container provides at runtime	Servlet API (Tomcat provides it)
runtime	Only needed at runtime	JDBC driver

8. Maven Repositories

Repository Type	What it is	Location
Maven Central	Public repo with millions of JARs	repo.maven.apache.org (internet)

Repository Type	What it is	Location
Local (~/.m2)	Cache on your machine — checked first	~/.m2/repository/ on your computer
Remote / Private	Company-internal repo (Nexus, Artifactory)	Custom URL configured in pom.xml

Download flow:

Step	Action
1	Maven reads pom.xml — identifies which dependencies are needed
2	Checks local cache: ~/.m2/repository/ — already downloaded?
3	If yes → use local copy (fast). If no → download from Maven Central / custom remote
4	After downloading → save copy to ~/.m2/repository/ for future use

First build is slow (downloads everything from internet). Subsequent builds are fast (uses local cache ~/.m2). Deleting ~/.m2/repository/ forces a full re-download.

Override repository URL (use company repo):

```
company-repo https://repo.coderarmyarmy.in/maven
```

9. POM Inheritance — Super POM & Effective POM

Just like Java classes inherit from Object, every pom.xml inherits from a Super POM. Spring Boot adds extra parent layers on top of that.

POM Layer	Purpose
Super POM	Root parent of all POMs — defines default plugins, repo URLs, etc.
spring-boot-dependencies	Defines all Spring library versions (BOM = Bill of Materials). Version alignment.
spring-boot-starter-parent	Inherits from dependencies — adds default plugin configs and encoding settings.
Your pom.xml (with <tag>)	Inherits from starter-parent — you just add your own deps and config.

Effective POM = Super POM + all parent POMs + your pom.xml merged. View in IntelliJ: right-click pom.xml → Maven → Show Effective POM.

Spring Boot parent declaration:

```
org.springframework.boot spring-boot-starter-parent 3.x.x
```

With this parent, you don't need to specify versions for any Spring libraries — they're all pre-aligned by the parent POM.

10. Maven Build Lifecycle

Maven has 3 separate lifecycle types:

Lifecycle	Phases	Use Case
clean	clean	Delete target/ folder — remove old build artifacts
default	validate → compile → test → package → verify → install → deploy	Main build pipeline from source to deployment
site	site	Generate HTML project documentation and reports

Running any phase automatically runs ALL previous phases first.

Default lifecycle — all phases explained:

Phase	What Happens
validate	Verify pom.xml is valid, required info present, project structure correct.
compile	Compile .java → .class files. Output goes to target/classes/
test	Run unit tests (JUnit/Mockito). Build stops here if any test fails.
package	Bundle compiled code into JAR/WAR. Stored in target/myapp-1.0.jar
verify	Run integration tests + code quality plugins (Checkstyle, JaCoCo, etc.)
install	Copy JAR to local ~/.m2/repository/ — available to other local Maven projects.
deploy	Upload JAR to remote/company repository — available to the whole team.

Common Maven commands:

Command	What it does
mvn validate	Validate project structure only
mvn compile	validate + compile all .java files
mvn test	validate + compile + run all tests
mvn package	validate + compile + test + create JAR in target/
mvn install	...all above + copy JAR to ~/.m2/ local repo
mvn clean	Delete target/ folder (clean slate)
mvn clean install	MOST COMMON — clean old build + fresh install to local repo
mvn clean compile	Clean then recompile from scratch
mvn deploy	Full lifecycle + upload JAR to remote repository

mvn clean install is the most frequently used command in real projects. Always use it to ensure a clean, reproducible build.

11. Maven Archetypes — Project Templates

When creating a new project in IntelliJ you can choose an archetype — a pre-built template that gives you starter code and dependencies already configured.

Archetype	What You Get
maven-archetype-quickstart	Simple Java app with JUnit pre-added. Basic Hello World.
spring-boot-web-starter (Maven Central)	Spring Boot web project skeleton — same as Spring Initializr output.
Company custom archetype	Your company's standard project template with internal standards pre-baked.

Spring Initializr (start.spring.io) is essentially a web-based Maven archetype generator.

Quick Reference — Key Concepts

Concept	One-line Explanation
JAR	Zipped package of compiled .class files + resources
pom.xml	Maven's config file — project info + dependencies + plugins

Concept	One-line Explanation
GAV	Group + Artifact + Version — unique ID for any JAR in the world
Dependency	External library listed in pom.xml that Maven downloads for you
Transitive Dependency	A dependency's own dependencies — Maven handles them automatically
Local repo (~/.m2)	Your machine's downloaded JARs cache — checked before internet
Maven Central	Public internet repository with millions of JARs
Super POM	Root parent POM — all project POMs inherit default config from it
Effective POM	Final merged POM that Maven actually uses (your POM + all parents)
Lifecycle phase	A step in the build: validate/compile/test/package/install/deploy
mvn clean install	Most used command — clean old build + build fresh + install locally
Archetype	Pre-built project template with starter code and deps ready to go

Next Lecture: Spring Core — IoC Container, Beans, Dependency Injection